

● QNX Neutrino RTOS

Raspberry Pi 4

Edge Computing

Predictive Maintenance at the Edge

Leveraging QNX RTOS Microkernel Architecture on Raspberry Pi 4 for deterministic, real-time industrial fault detection.

PROCESS 1

Real-time Deterministic
Sensor Fusion

PROCESS 2

Edge AI / DSP Math
Linear Regression

PROCESS 3

Asynchronous Cloud
Telemetry over UDP

Three Isolated QNX Processes

Each process runs in its own memory-protected address space, communicating exclusively through synchronous QNX message passing.



1 Sensor Core

Runs as **root**. Maps hardware memory directly. Polls MPU-6500 (I2C), DHT11 (GPIO bit-bang), SW420 (GPIO) every **500ms**.

- I2C
- GPIO
- mmap

2 Predictive Gateway BRAIN

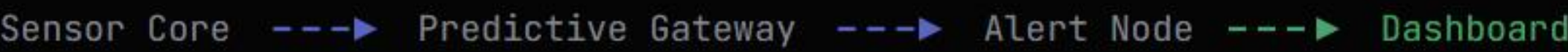
Polls Sensor Core every **5s**. Executes linear regression over 12-sample sliding window. Evaluates slope thresholds.

- DSP Math
- Regression
- Thresholds

3 Alert Node NETWORKER

Receives alert states from Gateway. Formats JSON payloads. Sends UDP datagrams to the Node.js dashboard.

- JSON
- UDP
- Async I/O



QNX Boot & Server Initialization

How the microkernel brings services online and prepares the IPC channel.

1 Microkernel Launch

`procnto` starts — initializes the MMU, Thread Manager, and Timer services. No device drivers, no filesystem, just the bare scheduler.

2 Service Registration via `name_attach()`

Sensor Core and Alert Node launch first. Each calls `name_attach(NULL, "sensor_core", 0)` and `name_attach(NULL, "alert_node", 0)` to register their IPC channel with the QNX name server (`resmgr`).

3 Block on `MsgReceive()`

Both servers enter a blocked state inside `MsgReceive()`, consuming 0% CPU. They remain dormant until the Predictive Gateway initiates the first transaction.

4 Gateway Connects via `name_open()`

The Predictive Gateway launches last. It calls `name_open("sensor_core", 0)` to obtain a connection ID, then fires the first `MsgSend()` — igniting the entire system loop.

Key Insight: The server processes are ready before the client exists. This is standard QNX design — servers attach names, clients discover them. Order of launch is intentional, not coincidental.

The Sensor Core

A dedicated hardware thread polls all three sensors at 500ms intervals, shielded from IPC interference by a POSIX mutex.

SENSOR_CORE.C – HARDWARE_THREAD

```
void *hardware_thread(void *arg) { while (running) { //
  Read MPU-6500 via I2C accel_x = read_mpu_axis(I2C_FD,
  0x3B); accel_z = read_mpu_axis(I2C_FD, 0x3F); // Read DHT11
  via GPIO bit-bang temp = dht11_read_temp(DHT_PIN); hum =
  dht11_read_hum(DHT_PIN); // Read SW420 vibration (digital)
  vib = gpio_read(SW420_PIN); // Critical section – lock
  shared state pthread_mutex_lock(&state_mutex);
  current_state.accel_x = accel_x; current_state.accel_z =
  accel_z; current_state.temp = temp; current_state.hum =
  hum; current_state.vib = vib;
  pthread_mutex_unlock(&state_mutex); delay(500); // 500ms
  poll cycle } return NULL; }
```

WHY THE MUTEX?

When the Gateway sends a **MsgSend**, the Sensor Core's IPC thread wakes up and reads **current_state**. Without the mutex, the hardware thread could be mid-write — sending partially-corrupted data (e.g., new X but old Z).

SENSOR DETAILS

MPU-6500	I2C @ 0x68
DHT11	GPIO 4 (bit-bang)
SW420	GPIO 17 (digital)

THREAD MODEL

Two threads within one process: **hardware_thread** (500ms polling loop) and the **main IPC thread** (blocks on **MsgReceive**). Both share **current_state** via the mutex.

Synchronous IPC — The Heartbeat

The fundamental building block of every QNX RTOS system. No queues, no signals, no shared memory — just blocking send/receive/reply.

MsgSend()

BLOCKED

Gateway calls MsgSend and is put to sleep by the kernel. CPU usage drops to exactly zero. No polling. No spinlocks.

MsgReceive()

RUNNING

Sensor Core unblocks, inherits Gateway's priority (priority inheritance), copies locked state into the reply buffer.

MsgReply()

READY

Sensor Core replies with sensor_packet_t. Gateway unblocks with fresh data — guaranteed consistent, no race conditions.

```
// Gateway side — the 5-second heartbeat while (running) {  
MsgSend(sensor_coid, &msg, sizeof(msg), &packet, sizeof(packet)); //  
packet now contains fresh, mutex-protected data  
process_prediction(&packet); delay(5000); }  

```

Why Not Pipes / Sockets? QNX MsgSend/Receive/Reply is a single kernel call that handles both data transfer and synchronization. No intermediate buffers, no copy steps, no priority inversions. The kernel guarantees the receiver runs at the sender's priority the instant the message is sent.

The Predictive Gateway

We don't just read data — we compute trend slopes to predict failure before it happens.

GATEWAY.C — PREDICTIVE MATH

```
// 12-sample sliding window linear regression float
calc_slope(float *y, int n) { float sum_x=0, sum_y=0,
sum_xy=0, sum_x2=0; for (int i=0; i<n; i++) { sum_x +=
(float)i; sum_y += y[i]; sum_xy += (float)i * y[i]; sum_x2
+= (float)i * (float)i; } float denom = n * sum_x2 - sum_x
* sum_x; return (n * sum_xy - sum_x * sum_y) / denom; } //
Evaluate thresholds slope_z = calc_slope(hist_z,
HISTORY_SIZE); slope_temp = calc_slope(hist_temp,
HISTORY_SIZE); if (slope_z > 5.0) status =
PREDICTIVE_WARNING; else if (slope_z > 10.0 || slope_temp >
2.0) status = CRITICAL; else status = NOMINAL;
```

SLIDING WINDOW

We maintain a circular buffer of **HISTORY_SIZE = 12** samples (60 seconds of data at 5s intervals). Each new reading pushes the oldest out.

WHY SLOPE, NOT THRESHOLD?

Hard limits trigger too late — vibration is already dangerous. By computing the **rate of change** (slope), we detect that vibration is *accelerating* toward failure, giving operators a predictive warning window.

THRESHOLD LOGIC

- `slope_z < 5.0` Nominal — normal operation
- `slope_z > 5.0` Predictive Warning — trend rising
- `slope_z > 10.0` Critical — imminent failure

The Alert Node

Network I/O is inherently non-deterministic. This process isolates it from the critical prediction loop.

ALERT_NODE.C – IPC RECEIVE + REPLY-FIRST PATTERN

```
while (running) { rcvid = MsgReceive(chid, &msg,
sizeof(msg), NULL); // *** CRITICAL: Reply immediately ***
MsgReply(rcvid, EOK, NULL, 0); // Gateway is already
unblocked and back // to its 5s math loop. We work async.
// Format JSON payload snprintf(json_buf, BUF_SZ, "
{"status":%d,\"z\":%.2f,\"
temp\":%.1f,\"slope_z\":%.3f}", msg.status, msg.accel_z,
msg.temp, msg.slope_z); // UDP send – may block, we don't
care sendto(udp_fd, json_buf, strlen(json_buf), 0, (struct
sockaddr*)&dest, sizeof(dest)); }
```

THE REPLY-FIRST PATTERN

The single most important architectural decision in this process. By calling **MsgReply(rcvid, EOK, NULL, 0)** **before** any JSON formatting or socket work, the Gateway is instantly unblocked.

WHAT THIS PREVENTS

If the network is congested, **sendto()** could block for hundreds of milliseconds. Without the reply-first pattern, the Gateway would be stuck waiting — missing its 5s prediction cycle, breaking determinism.

MEMORY SAFETY

Since the Alert Node is a separate QNX process, a crash in JSON formatting or a socket error **cannot** corrupt the Gateway's memory space. The microkernel enforces this at the hardware MMU level.

The Dashboard

Node.js catches UDP datagrams and pushes real-time telemetry to the browser — no polling, no refresh.

BACKEND

- Node.js **dgram** socket listens on UDP port **9876**
- Parses incoming JSON into typed objects
- Broadcasts to all connected browser clients via **Server-Sent Events**
- Zero-latency push — browser doesn't ask, server tells

FRONTEND

- **Chart.js** line graphs update in real-time — accel Z, temperature, slope trends
- System status badge updates dynamically without page refresh
- DOM mutations kept minimal for performance

localhost:3000 — Predictive Maintenance Dashboard

PREDICTIVE WARNING

Z-AXIS ACCELERATION



TEMPERATURE (°C)



CONCLUSION

Why QNX?

Because in predictive maintenance, a missed deadline isn't a dropped frame — it's a missed failure.

✗ Standard Linux

- CFS scheduler — no timing guarantees
- Thread starvation under I/O load
- Network stack can block kernel paths
- Shared memory — one crash kills all
- Non-deterministic interrupt latency

✓ QNX Neutrino RTOS

- Strict priority scheduling — guaranteed order
- Priority inheritance prevents inversion
- Network in separate process — isolated
- MMU-protected processes — crash-safe
- Bounded, deterministic interrupt latency

The Bottom Line

By utilizing the QNX microkernel, synchronous message passing, and priority inheritance, our edge node guarantees that mathematical evaluation occurs *exactly* when required — completely isolated from hardware hiccups, sensor stalls, or network dropouts. This is what real-time means.

○ Microkernel Architecture

○ Deterministic IPC

○ Fault Isolation